Présentation de l'application

La startup dédiée au coaching sportif

Plan de la présentation	
Rappel du contexte	**Page 2**
Gestion des fichiers et des composants React	**Page 3**
Les composants React	**Page 4**
Présentation du code et du visuel de l'application	**Page 5**
Observations	**Page 6**
Bilan	**Page 7**

Rappel du contexte

Bonjour,

En tant que développeur chez SportSee, startup dédiée au coaching sportif. Elle lance une nouvelle version de la page profil de l'utilisateur.

Mon objectif :

Je dois principalement me centrer sur le développement front-end et l'intégration avec l'API

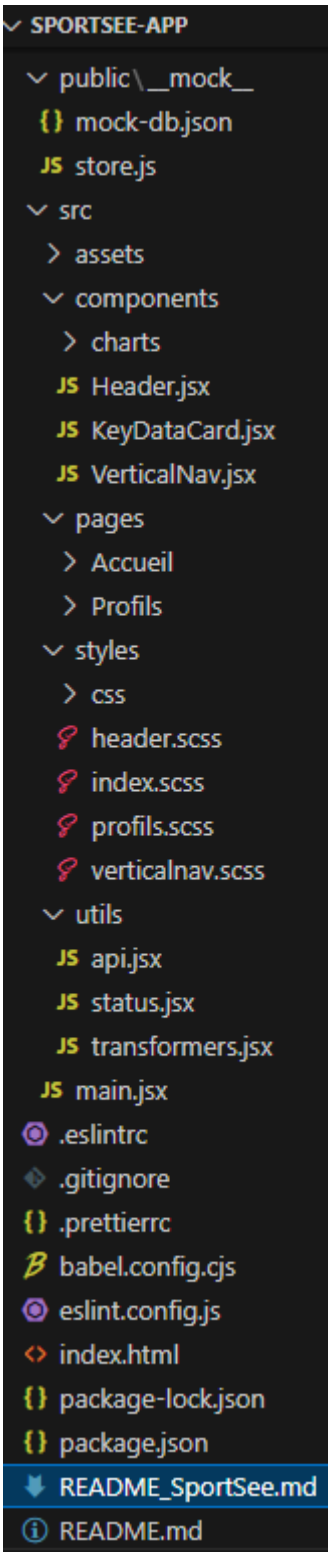
Démarrer le projet en utilisant React et développer la page profil de l'application pour desktop et tablette (responsive 1024x768), intégrant des graphiques sur l'activité sportive des utilisateurs.

Je disposais pour cela :

- d'un kanban contenant le sprint des US de la partie TODO à coder
- de contraintes techniques sous la forme de charts Recharts.D3
- d'un design de la nouvelle maquette sous Figma ainsi que du prototype de la page.
- d'un BackEnd à monter sur le port 3000 contenant les données de 2 utilisateurs
- de contraintes fonctionnelles concernant des appels asynchrones utilisant des bibliothèques comme Fetch ou Axios et des données mockées si le BackEnd est absent pour récupérer les données nécessaires.

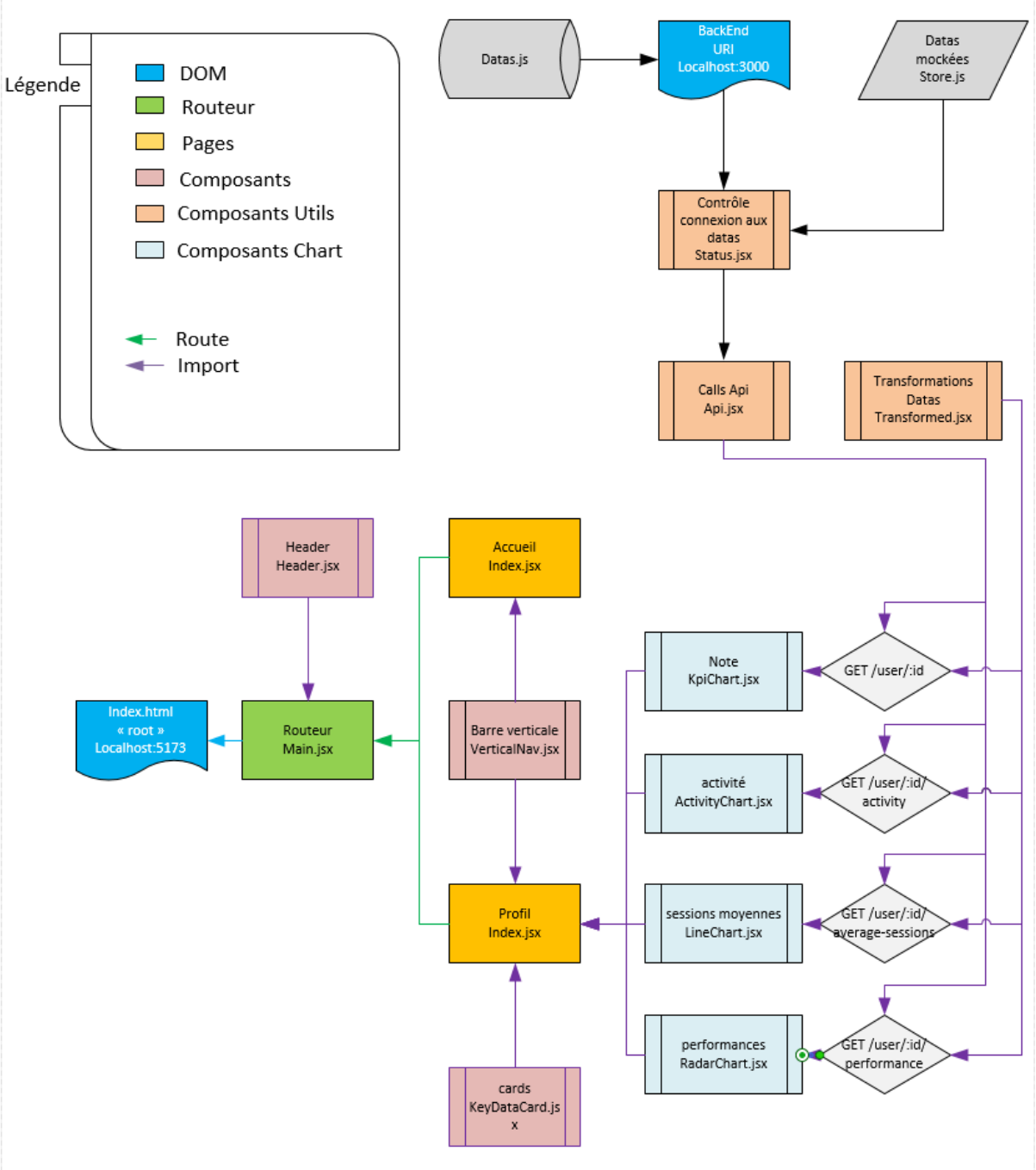
Objectif global : *Afficher des tableaux de bord dynamiques à partir de données d'une API (ou données locales en fallback).*

Gestion des fichiers et des composants React



Descriptions structure des fichiers
• Données mockées dans public/__mock__/ • Images et logo dans src/assets • Les calls Api, la transformation des datas et le status de la source dans src/utils/ • Les composants réutilisables dans src/components • Les composants recharts dans src/components/charts • Les composants pages dans src/pages • Les styles dans src/styles/css • Le saas à la racine de src/styles • Un fichier README_SportSee explicatif

Les composants React



Présentation du code et du visuel de l’application…

Observations

Utilisation d'Axios :
J'ai préféré Axios à fetch car sa syntaxe est plus claire et il gère automatiquement certains aspects utiles, comme la conversion JSON et la gestion des erreurs HTTP (404, 500, etc.). C'est un choix qui facilite la maintenance sur des projets amenés à évoluer.

Données mockées en fallback :
Un système de surveillance du backend a été mis en place. En cas d'indisponibilité, l'application bascule automatiquement sur des données locales placées dans le dossier public.

Les commentaires sont en français dans le code pour cette présentation mais peuvent être traduits en anglais si le besoin se fait sentir.

Difficultés rencontrées :

JSON Server :
J'ai tenté d'utiliser json-server pour simuler une API, mais j'ai rencontré des conflits de ports, et malgré le changement de port, les données restaient inaccessibles. Par souci d'efficacité, j'ai choisi de m'appuyer sur les données locales.

Différences entre Recharts 3D et autres libs :
Je me suis rapidement orienté vers Recharts pour sa simplicité. Les bibliothèques de graphiques 3D classiques demandent une mise en œuvre beaucoup plus complexe, peu adaptée à ce projet.

Bilan

Ce second projet en React m'a permis d'enrichir grandement mes connaissances sur les APIs et d'en apprendre plus sur les la librairie graphique et la manière d'y intégrer les styles et les données.

Le projet est structuré de manière claire :

- **Composants réutilisables** (charts, cards, etc.)
- **Données séparées** (données locales accessibles via /public)
- **Surveillance du BackEnd** dans un module dédié (status.js)
- **Rendu conditionnel** basé sur la disponibilité de l'API

Points forts du projet : robustesse face à une API instable, clarté du code.

Choix techniques réfléchis (Axios, fallback, Recharts).

Préparé pour l'évolutivité.